



Loops

Doing Something Again, Without Repeating Yourself

Session 6



What We're Covering Today



Why loops exist — the for loop, and iterating lists/tuples/strings



range(), enumerate(), and zip()



The while loop, and avoiding infinite loops



Input validation & sentinel-controlled loops



break, continue, and the loop else clause



Nested loops



Real-world patterns: accumulator, search, comprehensions



A second look at time complexity — nested loops



Mini projects tying everything together



PART 1

The for Loop

This is the moment lists and tuples finally pay off.



Why Do Loops Exist?



**Stop
repeating
yourself**

- Without a loop, printing 6 student names means writing `print()` six separate times
- A loop lets you say "do this once for every item" — one instruction, applied automatically to each item
- This is exactly why lists and tuples exist — a loop is how you actually use everything inside one
- Programmers call this principle DRY: Don't Repeat Yourself



The for Loop — Basic Syntax

- `for item in a_list:` runs the indented block once for every item in the list
- You choose the name "item" — it can be anything you like
- No indexing needed at all — Python hands you each value directly

```
fruits = ["apple", "banana", "mango"]
```

```
for fruit in fruits:  
    print(fruit)
```

```
# apple
```

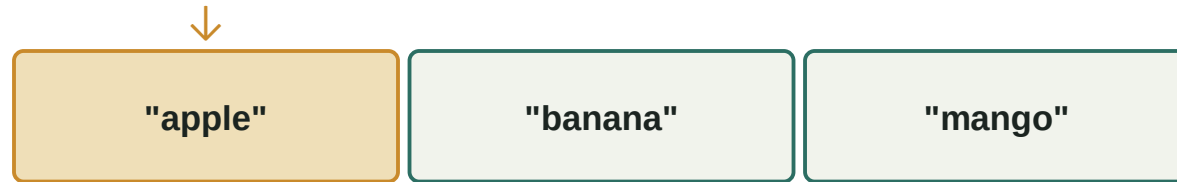
```
# banana
```

```
# mango
```



What's Actually Happening

for fruit in fruits: -> fruit takes each value, one at a time



- Round 1: fruit = "apple" Round 2: fruit = "banana" Round 3: fruit = "mango"
- The loop automatically moves to the next box each time, and stops after the last one
- You never manage the position yourself — that's the whole point



Looping Over Tuples and Strings

- The exact same for syntax works on a tuple — no changes needed
- It even works on a string, looping over one character at a time
- If you can index it or slice it, you can almost always loop over it too

```
coordinates = (4, 7, 9)
for number in coordinates:
    print(number)
```

```
for letter in "abc":
    print(letter)
```

```
# a
```

```
# b
```

```
# c
```



range() — Looping a Specific Number of Times

- range() generates a sequence of numbers — perfect when you want to repeat something N times
- range(n) counts from 0 up to (not including) n
- range(start, stop) and range(start, stop, step) give you full control

```
for i in range(5):
```

```
    print(i)      # 0 1 2 3 4
```

```
for i in range(2, 6):
```

```
    print(i)      # 2 3 4 5
```

```
for i in range(10, 0, -2):
```

```
    print(i)      # 10 8 6 4 2 <- counting down
```



for i in range(len(list)) — Index-Based Looping

- Sometimes you need the index itself, not just the value — this pattern gives you that
- Useful when you need to compare or update items at specific positions
- But if you only need the values, plain for item in list is simpler — don't reach for this by default

```
scores = [88, 92, 79]

for i in range(len(scores)):
    print("Position", i, "->", scores[i])

# Position 0 -> 88
# Position 1 -> 92
# Position 2 -> 79
```



enumerate() — Index and Value Together

- enumerate() gives you both the index and the value, in one clean loop — no range(len(...)) needed
- This is the preferred way to get both, and shows up constantly in real code

```
scores = [88, 92, 79]
```

```
for i, score in enumerate(scores):  
    print("Position", i, "->", score)
```

```
# Position 0 -> 88
```

```
# Position 1 -> 92
```

```
# Position 2 -> 79
```



zip() — Looping Over Two Lists Together

- zip() pairs up items from two (or more) lists, position by position
- Perfect for exactly the kind of data you worked with in the Grade Report mini project
- Stops automatically at the shorter list, if the lists are different lengths

```
names = ["Aisha", "Bilal", "Sara"]
scores = [88, 92, 79]

for name, score in zip(names, scores):
    print(name, "scored", score)

# Aisha scored 88
# Bilal scored 92
# Sara scored 79
```



Practice Set — for Loops

Try these yourself first — no code shown, that's the point

- 1 Loop over a list of 3 colors and print each one
- 2 Use `range(5)` to print the numbers 0 through 4
- 3 Use `range()` to print only the even numbers from 0 to 10
- 4 Loop over a string and print each character on its own line
- 5 Use `enumerate()` to print each item in a list along with its position
- 6 Use `zip()` to loop over two short lists of your own and print them paired together



PART 2

The while Loop



The while Loop — Basic Syntax

- while keeps running its block as long as a condition stays True
- Unlike for, it doesn't know in advance how many times it will run
- Something inside the loop must eventually make the condition False, or it never stops

```
count = 1

while count <= 5:
    print(count)
    count += 1

# 1 2 3 4 5
```



for vs. while — Which One?



Use for When...

- You already know how many times to repeat
- You're working through a list, tuple, or string
- Example: printing every item in a list



Use while When...

- You don't know in advance how many times you'll repeat
- You're waiting for a condition to change
- Example: keep asking until the user enters a valid answer



Infinite Loops — What Causes Them

- If the condition never becomes False, the loop never stops — this is an infinite loop
- The most common cause: forgetting to update the variable the condition depends on
- If your program ever seems frozen, this is the first thing to check

```
# BROKEN — count never changes, runs forever
```

```
count = 1
```

```
while count <= 5:
```

```
    print(count)
```

```
# FIXED — count is updated every time
```

```
count = 1
```

```
while count <= 5:
```

```
    print(count)
```

```
    count += 1
```



Input Validation Pattern

- A very common real use of while: keep asking until the user gives a genuinely valid answer
- The loop repeats the question every time the input fails the check

```
age = int(input("Enter your age: "))
```

```
while age < 0:
```

```
    print("Age cannot be negative.")
```

```
    age = int(input("Enter your age: "))
```

```
print("Thanks! Your age is", age)
```



Sentinel-Controlled Loops

- A sentinel value is a special input that means "stop the loop" — like typing "quit"
- This pattern is used everywhere: menus, chat programs, simple games

```
command = ""

while command != "quit":
    command = input("Type a command (or 'quit'): ")
    print("You typed:", command)

print("Goodbye!")
```



Practice Set — while Loops

Try these yourself first — no code shown, that's the point

- 1 Use a while loop to print the numbers 1 through 5
- 2 Use a while loop to count down from 10 to 1
- 3 Write a while loop that keeps asking for a password until the user types "python123"
- 4 Deliberately write an infinite loop, run it, then stop it with Ctrl+C



PART 3

Loop Control



break — Exit Early

- break immediately stops the loop completely, even if it wasn't finished
- Commonly used once you've found what you were looking for — no reason to keep going

```
numbers = [4, 8, 15, 16, 23, 42]
```

```
for n in numbers:
```

```
    if n == 15:
```

```
        print("Found it!")
```

```
        break
```

```
    print("Checking", n)
```



continue — Skip to the Next Round

- continue skips the rest of the current round only, then moves on to the next one
- The loop keeps going overall — only that one round is cut short

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
for n in numbers:
```

```
    if n % 2 == 0:
```

```
        continue    # skip even numbers
```

```
    print(n)
```

```
# 1 3 5
```



The else Clause on a Loop

- This one is genuinely Python-specific: a loop can have its own else block
- The else only runs if the loop finished normally — never triggered by break
- Useful for "search and report if nothing was found" patterns

```
numbers = [4, 8, 15, 16, 23, 42]

for n in numbers:
    if n == 100:
        print("Found it!")
        break
else:
    print("100 was never in the list")
```



Nested Loops

- A loop can contain another loop inside it — the inner loop runs completely for every single round of the outer one
- This is exactly how you work through a nested list (a grid) from last session

```
grid = [[1, 2, 3], [4, 5, 6]]
```

```
for row in grid:  
    for value in row:  
        print(value)
```

```
# 1 2 3 4 5 6
```



Practice Set — Loop Control

Try these yourself first — no code shown, that's the point

- 1 Loop through a list of numbers and use break to stop as soon as you find one over 50
- 2 Loop through a list of numbers and use continue to print only the odd ones
- 3 Write a for/else that searches for a name in a list and reports if it wasn't found
- 4 Write a nested loop that prints every item in a small 2x2 grid (list of lists)



PART 4

Patterns You'll Actually Use

The same handful of loop shapes show up again and again in real code.



The Accumulator Pattern

- Start with an empty value (0, "", or []), then build it up one step at a time inside the loop
- This is how sum(), and many things sum() can't do alone, actually work under the hood

```
numbers = [4, 8, 15, 16, 23, 42]
```

```
total = 0
```

```
for n in numbers:
```

```
    total += n
```

```
print(total) # 108
```



The Search Pattern

- Loop through everything, check a condition, and break the moment you find a match
- This is exactly what in secretly does behind the scenes

```
names = ["Aisha", "Bilal", "Sara"]  
target = "Bilal"
```

```
for name in names:  
    if name == target:  
        print("Found", target)  
        break
```



List Comprehensions — The Shortcut Version

- A list comprehension builds a new list from a loop, in a single line
- It's just a for loop that creates a list — same idea, more compact
- Use whichever version reads more clearly to you; both are completely valid

```
# the loop version
```

```
squares = []
```

```
for n in range(5):
```

```
    squares.append(n ** 2)
```

```
# the comprehension version — identical result
```

```
squares = [n ** 2 for n in range(5)]
```

```
print(squares) # [0, 1, 4, 9, 16]
```



Gotcha: Modifying a List While Looping

- Removing or adding items to a list while you're looping over it causes items to be skipped or repeated
- Python doesn't stop you — it just silently produces the wrong answer
- The safe fix: loop over a copy instead, with `list(original)` or `original[:]`

```
# BROKEN — skips an item because indices shift mid-loop
```

```
numbers = [1, 2, 3, 4]
```

```
for n in numbers:
```

```
    if n % 2 == 0:
```

```
        numbers.remove(n)
```

```
# SAFE — loop over a copy instead
```

```
for n in numbers[:]:
```

```
    if n % 2 == 0:
```

```
        numbers.remove(n)
```



A Second Look at Time Complexity



One Loop

Checks every item once — roughly n steps for n items



Nested Loops

Checks every pair — roughly $n \times n$ steps, much slower as the list grows



Practice Set — Patterns

Try these yourself first — no code shown, that's the point

- 1 Use the accumulator pattern to count how many numbers in a list are negative
- 2 Use the search pattern to check if a specific word exists in a list of words
- 3 Write a list comprehension that builds a list of the first 10 square numbers
- 4 Predict what a broken "modify while looping" example will print, then run it to check



Today's Tasks — Part A: for Loops

Small tasks — do them in order



1

Print every number from 1 to 10 using a for loop and range()



2

Loop through a list of 5 of your favorite movies and print each one



3

Use enumerate() to print a numbered list (1. ..., 2. ..., 3. ...) from a list of items



4

Use zip() to print 3 names next to 3 matching ages, paired together



Today's Tasks — Part B: while Loops & Control

Small tasks — do them in order



5

Write a while loop that adds up numbers the user enters, stopping when they type 0



6

Write a loop that searches a list for a number, using break once it's found



7

Write a nested loop that prints a small multiplication table (1 to 3, times 1 to 3)



PART C

Put It All Together

One project using everything from this course so far — lists, tuples, conditionals, and now loops.



Mini Project: Class Grade Report, Revisited



**Now with
a real loop**

- Take your Class Grade Report project from last session — the one built with direct indexing
- Rebuild it using a for loop with `zip()` to pair names and scores, instead of manual indexing
- Inside the loop, print each name, score, and Pass/Fail — the same output, far less code
- Use the accumulator pattern to compute the total and average yourself, instead of relying only on `sum()`



RECAP

**Everything you learned about lists and tuples
was leading here — one instruction, applied to
everything.**

Next session: strings in depth — indexing, slicing, and methods, now with loops to back them up.